

BDMH Assignment 2: RNA Classification Documentation

Group 45: Ayush Singh (2023163), Aryan Dutt (2023143), Aryan Dahiya (2023142)

April 16, 2026

1 Purpose

This standalone Python script provides a robust pipeline for miRNA (RNA) classification. It combines handcrafted biological features (structural stability, taxonomy, and sequence motifs) with high-dimensional sequence embeddings from the pretrained **RNA-FM** language model. The pipeline is designed to predict if a given RNA sequence is a functional miRNA (+1) or not (0). It utilizes an ensemble of gradient boosting models (LightGBM, CatBoost) and Logistic Regression, optimized via a multi-level stacking strategy to achieve high generalization performance. **Public AUC is 0.87.**

2 Imports and Libraries

The script depends on several specialized libraries for bioinformatics and machine learning:

- **ViennaRNA (RNA)**: Core library used for secondary structure prediction.
 - **multimolecule & transformers**: Interface for the **RNA-FM** foundation model.
 - **torch**: Backend for deep learning embedding extraction.
 - **catboost & lightgbm**: Primary gradient-boosted tree classifiers.
 - **scikit-learn**: Cross-validation, PCA, and metric utilities.
 - **pandas & numpy**: Standard data processing.
-

3 Command Line Input

The script `Group_45.py` is configured via CLI:

- `--input_path`: Directory containing CSV/FASTA inputs.
 - `--output_path`: Directory for submission outputs.
 - `--gpu`: Choice of `off`, `on`, or `auto`.
 - `--all`: Choice to save all ensemble variants or just the best.
-

4 Fuzzy Matching Strategy

A key innovation is the **Fuzzy Hamming Matching** against the miRBase database. This recovers signal for sequences that aren't exact matches but are biologically similar.

```
def fuzzy_match(query, max_dist=2):
    qlen = len(query)
    best_dist = max_dist + 1
    best_entries, best_seq = [], None

    # Try Hamming distance on same length and +/- 1 length indels
    for ref_seq, ref_names in mirbase_by_len.get(qlen, []):
        d = hamming_distance(query, ref_seq)
        if d < best_dist:
            best_dist, best_entries, best_seq = d, list(ref_names), ref_seq
        elif d == best_dist and d <= max_dist:
            best_entries.extend(ref_names)
    ...
```

5 Extracting RNA-FM Embeddings

For deep sequence context, the script uses the **RNA-FM** model. Mean activation pooling is applied across tokens, followed by PCA reduction for regularization.

```
def extract_frozen_embeddings(seqs, tokenizer, model, batch_size=64):
    ...
    with torch.no_grad():
        out = model(input_ids=b_ids, attention_mask=b_mask,
                    output_hidden_states=True)
    # Average last 4 layers for richer representation
    stacked = torch.stack(out.hidden_states[-4:], dim=0)
    pooled = (stacked.mean(dim=0) * mask).sum(1) / (mask.sum(1) + 1e-8)
    ...
```

6 Stacking and Meta-Ensembling

Final predictions are generated using multi-level stacking and rank-averaging to ensure robustness across different model families.

```
# Rank average (non-parametric, robust)
def rank_average(*arrays):
    ranks = [rankdata(a) / len(a) for a in arrays]
    return np.mean(ranks, axis=0)

# Weighted blend (grid search on OOF)
for w in np.arange(0.1, 0.95, 0.05):
    blend = w * oof_best_cpu + (1 - w) * oof_emb
    if roc_auc_score(y, blend) > best_blend_auc:
        best_blend_auc, best_w = blend_auc, w
```

7 Submission File Creation

The final output is `sub_v5_BEST.csv`, containing unique identifiers and predicted probability scores $[0, 1]$.